

Funzioni Ricorsive

Introduzione

Abbiamo visto come funzionano le [funzioni in Python](#) e quali sono i [vantaggi](#) nell'implementarle nel codice.

Esiste una particolare uso delle funzioni chiamate **funzioni ricorsive**.

Le funzioni ricorsive

Sono funzioni che chiamano sé stesse durante la loro esecuzione. == Questo approccio è particolarmente utile == per risolvere problemi che possono essere scomposti in sotto-problemi più piccoli dello stesso tipo, come ad esempio il calcolo del fattoriale, la sequenza di Fibonacci o la costruzione e la navigazione interna di strutture dati gerarchiche (es: alberi).

Caratteristiche principali

La logica delle funzioni ricorsive prevede 3 caratteristiche principali:

1. Caso base:

Ogni funzione ricorsiva deve avere almeno un caso base che interrompe la ricorsione.

2. Caso Ricorsivo:

La parte della funzione che chiama sé stessa con un input più semplice.

3. Avvicinamento al caso base:

Ogni chiamata ricorsiva deve avvicinarsi al caso base.

Prendiamo ad esempio il calcolo del fattoriale:



Python

```
1 def fattoriale(n:int)->int:
2     #caso base
3     if n == 0 or n<1:
4         raise ValueError("Il numero deve essere maggiore di
zero")
5     elif n == 1:
6         return 1
7     #caso ricorsivo
```

```

8         else:
9             return n * fattoriale(n-1)
10
11 print(fattoriale(5))

```

Analizziamo questo codice:

1. **Caso base:**

- `if n == 0 or n < 1:`
Restituisce un messaggio d'errore se `n` non è positivo.
- `elif n == 1:`
Restituisce `1`, terminando la ricorsione.

2. **Caso Ricorsivo:**

- `else: return n * fattoriale(n-1)`

3. **Avvicinamento al caso base:**

- `n-1:`
 - Ogni chiamata ricorsiva passa `n-1` invece di `n`, riducendo progressivamente il problema.
Infatti nel `print()` si va a chiamare la funzione `fattoriale()` e viene passato il valore `5` come argomento, viene calcolato il fattoriale di `5!` (cioè $5*4*3*2*1 = 120$) e quando viene raggiunto il caso base (`n == 1`) la ricorsione si ferma.

Difatti senza la diminuzione progressiva di `n` non ci si avvicinerebbe al caso base, tuttavia, benché la diminuzione progressiva sia importante, non è sufficiente:

serve prima di tutto un caso base affinché venga raggiunto e lo stack di chiamate si interrompa.

Per essere più precisi:



Python

```

1 def fattoriale(n: int) -> int:
2     if n == 0 or n < 1:
3         return f"Il numero inserito deve essere maggiore di zero e 1"
4     return n * fattoriale(n - 1) # Caso ricorsivo senza fermarsi mai
5
6 print(fattoriale(5))

```

In questo caso la funzione continua a chiamarsi all'infinito (`fattoriale(0)` , poi `fattoriale(-1)` , `fattoriale(-2)` , etc.) fino a che Python non dà errore di ricorsione perché lo stack di chiamate si riempie troppo.

Esempio di esecuzione per `fattoriale(5)` :

1. `fattoriale(5)` → `5 * fattoriale(4)`
2. `fattoriale(4)` → `4 * fattoriale(3)`
3. `fattoriale(3)` → `3 * fattoriale(2)`
4. `fattoriale(2)` → `2 * fattoriale(1)`
5. `fattoriale(1)` → **caso base**, ritorna 1

Ora, i valori si moltiplicano risalendo la catena:

`2 * 1 = 2` , `3 * 2 = 6` , `4 * 6 = 24` , `5 * 24 = 120` .

Esercizio con funzione ricorsiva: Countdown

Scrivi una funzione Python chiamata `countdown` che accetti un numero intero positivo `n` come input e stampi un conto alla rovescia da `n` a zero.

Se il numero di input è negativo, visualizza un messaggio di errore.

Per implementare la funzione, devi usare esclusivamente un ciclo `while` e il parametro `n` passato come input alla funzione.

Non è consentito dichiarare variabili aggiuntive all'interno della funzione.

Successivamente, chiama la funzione con `n = -5` e `n = 5` .

Expected Output:

Error! Inserted number is negative!

5
4
3
2
1
0

Normalmente per risolvere questo esercizio si usa un `while` loop:



Python

```
1 def countdown(n: int) -> None:
2     # n is positive (n >= 0)
```

```

3     if n >= 0:
4         # print the countdown, while n >= 0
5         while n >= 0:
6             print(n)
7             n = n - 1
8         # n is negative (n < 0)
9     else:
10        print("Error! Inserted number is negative!")
11
12 # calling function countdown for n = -5
13 countdown(-5)
14 # calling function countdown for n = 5
15 countdown(5)

```

Passaggi della funzione `countdown(n)`

- **Controllo del valore di `n`**
 - Se `n` è **positivo o zero** (`n >= 0`), esegue un **ciclo** `while` che stampa `n` e lo decrementa di 1 fino ad arrivare a `0`.
 - Se `n` è **negativo** (`n < 0`), entra nell'`else` e stampa un messaggio di errore.
- **Uso del ciclo `while`**
 - Il ciclo `while n >= 0`: continua a eseguire il corpo del ciclo finché `n` non diventa inferiore a `0`.
 - Ad ogni iterazione, `n` viene decrementato di 1 (`n = n - 1`).
 - Quando `n` diventa `-1`, la condizione `n >= 0` diventa falsa e il ciclo si interrompe.

Il flusso di esecuzione quindi sarà:

1. **Prima chiamata** (`countdown(-5)`):

- Poiché `-5 < 0`, esegue il blocco `else`
- Stampa: "Error! Inserted number is negative!"

2. **Seconda chiamata** (`countdown(5)`):

- Poiché `5 >= 0`, entra nel blocco `if`
- Esegue il ciclo `while`:
 - Stampa: 5
 - Decrementa: `n = 4`
 - Stampa: 4
 - Decrementa: `n = 3`

- ... continua finché $n \geq 0$
- Stampa finale: 0
- Esce dal ciclo quando n diventa -1

Se usassimo la funzione ricorsiva al posto del ciclo while:

Expected Output:

Error! Inserted number is negative!

5
4
3
2
1
0

Qui abbiamo la `def` della funzione:



Python

```
1 def countdown(n:int):  
2     #n è negativo  
3     if n<0:  
4         print(error)
```

L'if, in questo caso, impedisce che la funzione richiami se stessa all'infinito quindi prima di fare una chiamata ricorsiva dobbiamo costruire il codice che a monte ci sia una condizione per cui il codice si stoppa; ovvero il caso base.



Python

```
1 elif n==0:  
2     print(0)
```

L' `elif` è un'altra condizione di terminazioni delle ricorsioni(o caso base):

Quindi la chiamata deve terminare quando n è uguale a 0 o minore di 0,

Il caso ricorsivo lo mettiamo nell'else:



Python

```
1 else:
```

```
2         print(n-1)
```

Anche qui `n-1` è l' avvicinamento al caso base.

Quindi il codice completo è:



Python

```
1  def countdown(n:int) -> None:
2      # n is negative
3      if n < 0 :
4          print("Error! Inserted number is negative!")
5      # n = 0 must stop the recursive process
6      elif n == 0:
7          print(0)
8      # other cases
9      else:
10         print(n)
11         countdown(n-1)
12
13  countdown(0)
14  print("-----")
15  countdown(5)
16  print("-----")
17  countdown(20)
```

Passaggi logici del codice

Struttura della Funzione

1. **Caso base 1 ($n < 0$):**
 - Gestione dell'errore per input negativi
 - Stampa un messaggio di errore e termina
2. **Caso base 2 ($n == 0$):**
 - Condizione di terminazione della ricorsione
 - Stampa 0 e termina la catena ricorsiva
3. **Caso ricorsivo ($n > 0$):**
 - Stampa il valore corrente di n
 - Chiama se stessa con `n-1` (avvicinamento al caso base).

Flusso di esecuzione

Chiamata `countdown(0)`



Python

```
1 0
```

- Entra nel caso `elif n == 0`
- Stampa 0 e termina (nessuna ricorsione)

2. Chiamata `countdown(5)`



Python

```
1 5
2 4
3 3
4 2
5 1
6 0
```

- Sequenza ricorsiva:
 1. Stampa 5 → chiama `countdown(4)`
 2. Stampa 4 → chiama `countdown(3)`
 3. Stampa 3 → chiama `countdown(2)`
 4. Stampa 2 → chiama `countdown(1)`
 5. Stampa 1 → chiama `countdown(0)`
 6. Stampa 0 → termina (caso base raggiunto)

3. Chiamata `countdown(20)`

- Stampa i numeri da 20 a 0 in ordine decrescente
- Segue lo stesso pattern di `countdown(5)` ma con più passaggi

Differenza Rispetto alla versione iterativa

1. **Assenza di cicli:** Tutta la logica è gestita dalle chiamate ricorsive
2. **Stack di chiamate:** Ogni chiamata viene messa nello stack fino al raggiungimento del caso base
3. **Terminazione naturale:** La ricorsione si interrompe automaticamente quando `n` raggiunge 0

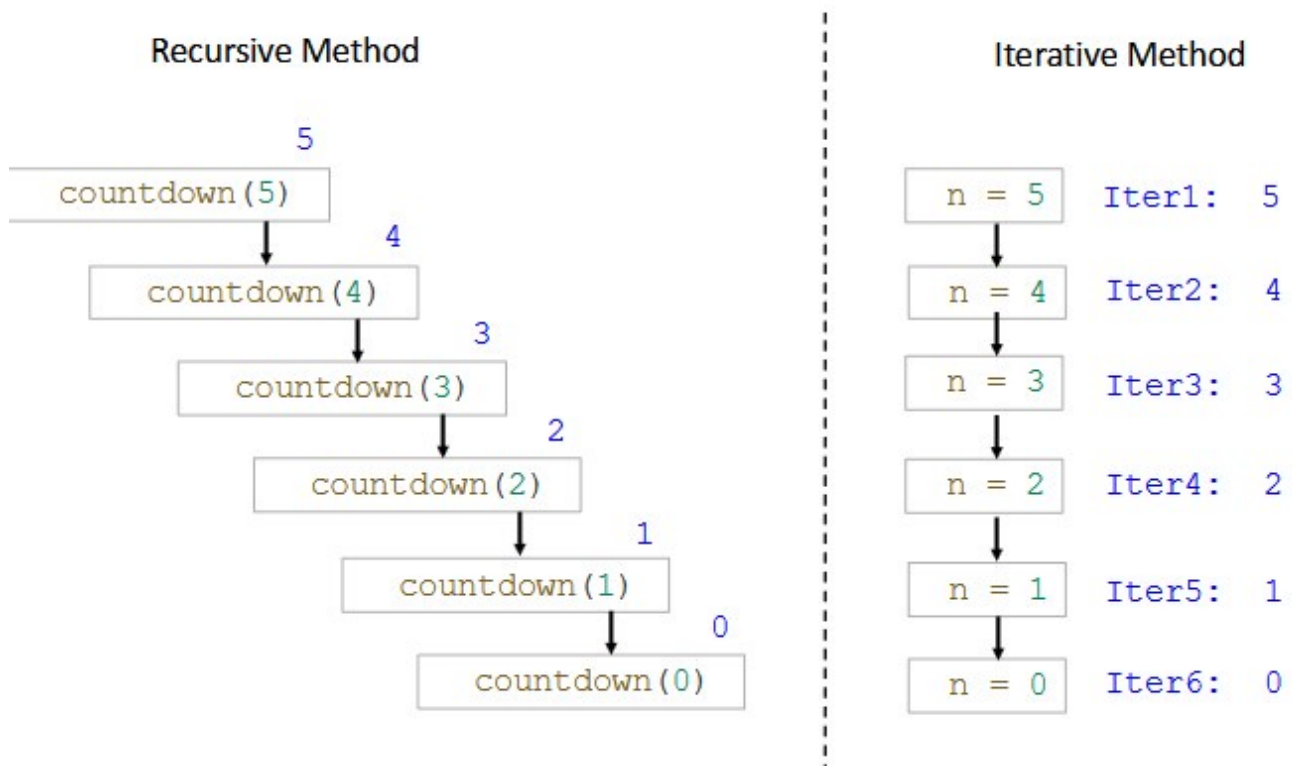
☰ Regola generale:

- **Blocchi** `if / elif` → **Definiscono i casi base (terminazione della ricorsione).**
- **Blocco** `else` → **Contiene la chiamata ricorsiva per continuare il processo.**

Un ultimo consiglio è quello di usare sempre e solo il `return` e mai il `print()` all'interno della catena dei conditional statements

Metodo ricorsivo vs. metodo iterativo

Questo problema può essere risolto sia attraverso il metodo iterativo che con le funzioni ricorsive:



Come possiamo vedere dall'immagine nel **metodo ricorsivo** a sinistra:

- La funzione `countdown(n)` chiama sé stessa con un valore ridotto (`countdown(n-1)`) finché `n` non diventa `0`.
- Ogni chiamata genera un nuovo livello nella pila di esecuzione, che viene poi risolto tornando indietro.
- Struttura visiva:
una serie di chiamate annidate che vanno da `countdown(5)` fino a

`countdown(0)` , poi si risolvono tornando indietro.

Nel metodo iterativo a destra:

- Utilizza un ciclo che **decrementa** `n` **ad ogni iterazione** fino a raggiungere `0` .
- Non vengono create nuove istanze della funzione, quindi consuma meno memoria rispetto alla versione ricorsiva.
- Ogni iterazione è numerata (Iter1, Iter2, etc.), mostrando il valore di `n` a ogni passo.

Ovviamente con il metodo ricorsivo scrivere questo codice risulta più veloce da scrivere, più leggibile e più intuitivo. Tuttavia come abbiamo già accennato prima la ricorsione è un buon modo per costruire strutture gerarchiche ad albero ed esplorarle ma se l'albero è troppo profondo si rischia di aumentare il calcolo computazionale e di rallentare la macchina, infatti questo tipo di funzioni vengono usate in alcuni casi specifici quando servono.

Esercizio 2: Somma ricorsiva

Scrivi una funzione chiamata `sum` che prende un numero intero in input e ritorna da 0 a `n`.

Se l'input `n` è negativo, visualizza un messaggio di errore e la funzione deve ritornare 0.

Per implementare la funzione `sum`, devi esclusivamente usare un `while` loop e il parametro `n` passato in input nella funzione.

Expected Output:

```
1  Error! Inserted number is negative!
2
3  0
4
5  -----
6  15
```

1. Caso con il `while` loop:



Python

```
1  def sum(n:int) -> int:
2      # n is negative
3      if n < 0:
4          print("Error! Inserted number is negative!")
5          return 0
```

```

6     # n is positive
7     else:
8         # define a sum
9         sum:int = 0
10        # compute sum until n >= 0
11        while n>=0:
12            sum = sum + n
13            n = n-1
14        # return sum as int
15        return int(sum)
16    print(sum(-5))
17    print("-----")
18    print(sum(5))

```

Analizziamo questo codice:

1. Gestione dell'input negativo



Python

```

1  if n < 0:
2      print("Error! Inserted number is negative!")
3      return 0

```

- **Logica:** Se `n` è negativo:
 - Stampa un messaggio di errore
 - Restituisce 0 (come specificato nei requisiti)

2. Calcolo della somma per input positivo:



Python

```

1  else:
2      sum: int = 0
3      while n >= 0:
4          sum = sum + n
5          n = n - 1
6      return int(sum)

```

- **Inizializzazione:**
 - `sum = 0` - Variabile accumulatore per la somma
- **Ciclo while:**

- Condizione: `n >= 0` (continua finché `n` non diventa negativo)
- Ad ogni iterazione:
 - Aggiunge il valore corrente di `n` a `sum`
 - Decrementa `n` di 1
- **Ritorno del risultato:**
 - `return int(sum)` - Restituisce la somma convertita in intero (la conversione è ridondante poiché `sum` è già intero)

☰ Esempio di esecuzione >

Caso 1: Input negativo (`sum(-5)`)

1. Verifica `n < 0` → True
2. Stampa: "Error! Inserted number is negative!"
3. Restituisce: 0
4. Output:

```
1    Error! Inserted number is negative!
2    0
```

Caso 2: Input positivo (`sum(5)`)

1. Inizializza `sum = 0`
2. Esegue il ciclo:
 - Iterazione 1: `n=5` → `sum=0+5=5`, `n=4`
 - Iterazione 2: `n=4` → `sum=5+4=9`, `n=3`
 - Iterazione 3: `n=3` → `sum=9+3=12`, `n=2`
 - Iterazione 4: `n=2` → `sum=12+2=14`, `n=1`
 - Iterazione 5: `n=1` → `sum=14+1=15`, `n=0`
 - Iterazione 6: `n=0` → `sum=15+0=15`, `n=-1`
1. Esce dal ciclo quando `n=-1`
2. Restituisce: 15

Il ciclo `while` termina sempre perché `n` viene decrementato ad ogni iterazione

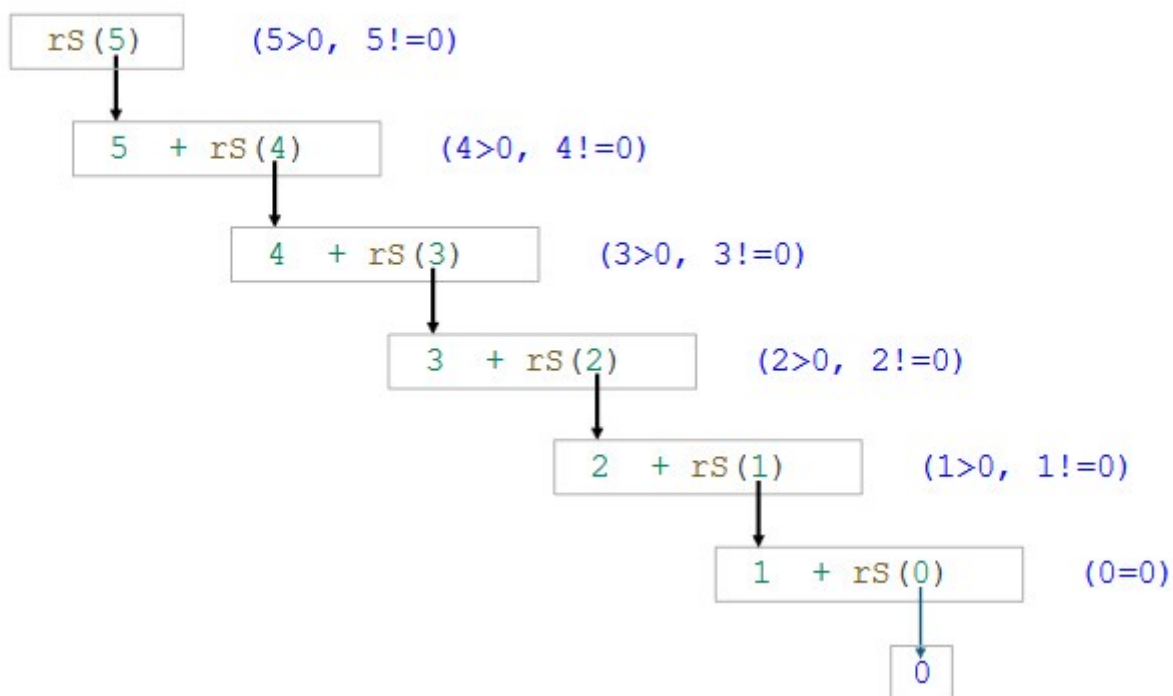
Ora anziché il `while` usiamo la funzione ricorsiva:



Python

```
1 def recursiveSum(n:int) -> int:
2     # n is negative
3     if n < 0:
4         print("Error! Inserted number is negative!")
5         return None
6     # if n = 0 recursive process must be stopped
7     elif n == 0:
8         return 0
```

la rappresentazione grafica è:

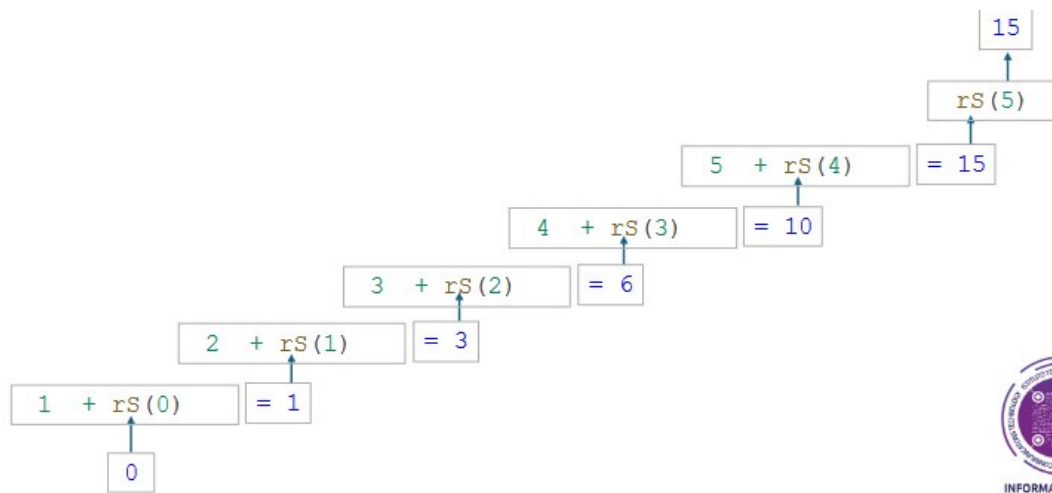


Osservando questa immagine possiamo vedere come funziona la funzione ricorsiva:

ho il numero intero 5 , ch     maggiore e diverso da zero
devo fare $5 + (n-1)$, cio  :

- $5 + (5-1)$, 4   maggiore e diverso da zero? Si
- $4+(4-1=3)$, 3   maggiore e diverso da zero ? Si
- $3+(3-1=2)$, 2 e maggiore e diverso da zero ? Si
- $2+(2-1= 1)$, 1   maggiore e diverso da zero ? Si
- $1+(1-1= 0)$, 0   maggiore e diverso da zero ? No, qui la ricorsione si stoppa.

Volendo possiamo anche vedere il flusso della funzione dal basso verso l'alto, cio   partendo da 0 .



Il procedimento è lo stesso solo che è inverso
Possiamo adesso scrivere la nostra funzione ricorsiva



Python

```

1  def recursiveSum(n:int) -> None:
2      # n is negative
3      if n < 0:
4          print("Error! Inserted number is negative!")
5          return None
6      # if n = 0 recursive process must be stopped
7      elif n == 0:
8          return 0
9      # if n is positive, compute recursive sum
10     else:
11         return int(n + recursiveSum(n-1))
12
13     print(recursiveSum(5))

```

Guarda immagine pag.34.

Esempio 3: Sum in range

Scrivi una funzione `sumInRange` che calcola la somma di tutti i numeri interi compresi tra `a` e `b`, inclusi, dove `a` e `b` vengono passati come input alla funzione.

Per risolvere l'esercizio, è obbligatorio utilizzare un ciclo `while`, e si assume che il valore di `b` sia sempre maggiore di `a`. Pertanto, se `a > b`, è necessario scambiare i valori per assicurarsi che `a` sia il più piccolo dei due.

Infine, è consentito dichiarare solo una variabile, oltre ai parametri della funzione, per memorizzare la somma.

Infine, chiama la funzione `sumInRange` per `a = 5, b = 10` e per `a = 10, b = 5`.

Expected Output:

```
1 45
2
3 ---
4 45
```

Quindi l'esercizio è:



Python

```
1 def sumInRange(a:int, b:int) -> int:
2     # if a > b, swap values of a and b
3     if a > b:
4         # define a temporary variable called temp, containing value
of a
5         temp:int = a
6         # swap values of a and b
7         a = b
8         b = temp # now b = a
9     # define a sum
10    sum:int = 0
11    # compute sum until b = a
12    while b>=a:
13        sum = sum + b
14        b = b -1
15    # return sum
16    return int(sum)
```

Analizziamo il codice:

1. Gestione dell'ordine degli estremi



Python

```
1 if a > b:
2     temp: int = a
3     a = b
4     b = temp
```

- **Logica:** Se `a` è maggiore di `b`, scambia i valori delle due variabili
- **Meccanismo:**
 - Usa una variabile temporanea `temp` per conservare il valore di `a`
 - Assegna a `a` il valore di `b`
 - Assegna a `b` il valore originale di `a` (conservato in `temp`).

2. Calcolo della somma



Python

```
1  sum: int = 0
2  while b >= a:
3      sum = sum + b
4      b = b - 1
5  return int(sum)
```

- **Inizializzazione:**
 - `sum = 0` - Variabile accumulatore per la somma
- **Ciclo while:**
 - Condizione: `b >= a` (continua finché `b` non diventa minore di `a`)
 - Ad ogni iterazione:
 - Aggiunge il valore corrente di `b` a `sum`
 - Decrementa `b` di 1
- **Ritorno del risultato:**
 - `return int(sum)` - Restituisce la somma (la conversione è ridondante poiché `sum` è già intero)

≡ Esempio di esecuzione

Caso 1: Input ordinato (`sumInRange(2, 5)`)

1. Non entra nell' `if` (`2 < 5`)
2. Esegue il ciclo:
 - Iterazione 1: `b=5` → `sum=0+5=5`, `b=4`
 - Iterazione 2: `b=4` → `sum=5+4=9`, `b=3`
 - Iterazione 3: `b=3` → `sum=9+3=12`, `b=2`
 - Iterazione 4: `b=2` → `sum=12+2=14`, `b=1`
3. Esce dal ciclo quando `b=1`

4. Restituisce: 14 (5+4+3+2)

Caso 2: Input non ordinato (`sumInRange(5, 2)`)

5. Entra nell' `if` (5 > 2) e scambia i valori:

- a diventa 2
- b diventa 5

1. Esegue lo stesso calcolo del Caso 1

2. Restituisce: 14 (stesso risultato)

Implementato con la funzione ricorsiva è:



Python

```
1 def recursiveSumInRange(a:int, b:int) -> int:
2     # if a > b, swap values of a and b
3     if a > b:
4         # define a temporary variable called temp, containing value
        of a
5         temp:int = a
6         # swap values of a and b
7         a = b
8         b = temp # now b = a
```

Nel caso in cui a e b siano uguali ritorno uno dei due valori:



Python

```
1 def recursive_sum_Range(a:int, b:int) -> int:
2     # if a > b, swap values of a and b
3     if a > b:
4         # define a temporary variable called temp, containing value
        of a
5         temp:int = a
6         # swap values of a and b
7         a = b
8         b = temp # now b = a
9         # if b = a, the recursive process must be stopped
10        if b == a:
11            return int(a)
12        # otherwise, compute the sum recursively
```

```

13         else:
14             return int(b + recursive_sum_range(a, b-1))
15
16     print(recursive_sum_range(5,10))

```

Analizziamo il codice:

1. Gestione dell'ordine degli estremi:



Python

```

1  if a > b:
2      temp: int = a
3      a = b
4      b = temp

```

- **Scambio intelligente:** Garantisce che `a` sia sempre $\leq$ `b` prima del calcolo
- **Effetto:** `range(a, b)` sarà sempre valido

2. Caso base:



Python

```

1  if b == a:
2      return int(a)

```

- **Terminazione:** Quando gli estremi coincidono, restituisce il valore stesso
- **Esempio:** `recursiveSumInRange(3, 3)` $\rightarrow$ 3

3. Caso Ricorsivo:



Python

```

1  else:
2      return int(b + recursiveSumInRange(a, b-1))

```

- **Logica ricorsiva:**
 - Somma il valore corrente `b`
 - Richiama la funzione con intervallo ridotto (`a, b-1`)
- **Esempio:**
 - `recursiveSumInRange(2, 4)` $\rightarrow$ `4 + recursiveSumInRange(2, 3)`
 - $\rightarrow$ `4 + (3 + recursiveSumInRange(2, 2))`

- $\rightarrow 4 + 3 + 2 = 9$

≡ Esempio di esecuzione

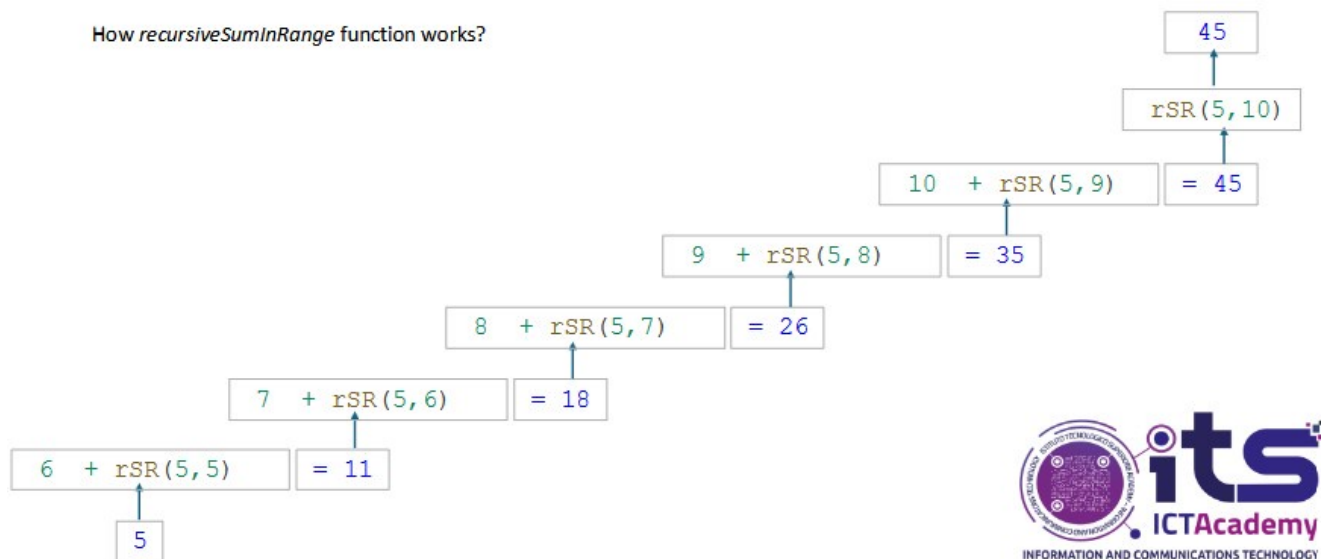
Input: `recursiveSumInRange(3, 5)`

1. Non scambia ($3 < 5$)
2. $5 + \text{recursiveSumInRange}(3, 4)$
 - $4 + \text{recursiveSumInRange}(3, 3)$
 - Caso base: 3
 - $4 + 3 = 7$
3. $5 + 7 = 12$
4. **Output:** 12 ($3+4+5$)

Input con scambio: `recursiveSumInRange(5, 3)`

5. Scambia: $a=3, b=5$
6. Segue lo stesso processo dell'esempio precedente

Guardando questa immagine possiamo capire come la funzione ricorsiva torni:



Partendo dal basso si ha l'intero 5 :

- $6+5=11$
- $11+7=18$
- $18+8=26$
- $26+9=35$

- 35+10=45

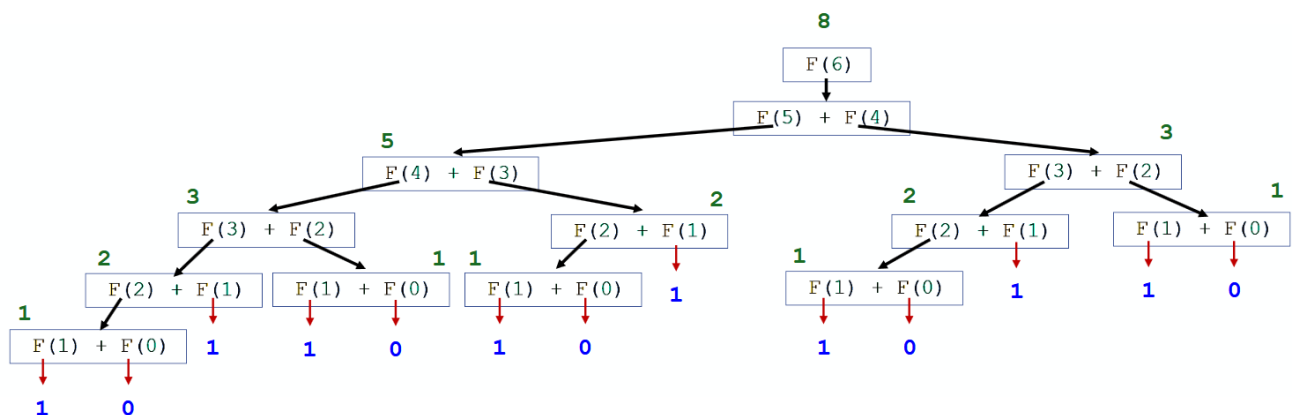
Fibonacci

La sequenza di Fibonacci è una sequenza di numeri interi dove ogni numero è la somma dei numeri che lo precedono e alla fine si va a creare quest' albero.

Formal definition

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Per ogni funzione devo eseguire questa formula finché non raggiungo 0 o 1 .
Per capire meglio Fibonacci, usiamo questa immagine:



Per implementare ciò in Python:



Python

```
1  def fibonacci(n:int) -> int:
2      # n is negative or 0
3      if n <= 0:
4          return 0
5      # n = 1
6      elif n == 1:
7          return 1
8      # otherwise, compute Fibonacci series applying its formula
9      else:
10         return int(fibonacci(n-1) + fibonacci(n-2))
```